AI 2002 Artificial Intelligence

Assignment # 2: GAME AI

Faculty: Ms. Umarah Qaseem

Assigned: 18 March 2026
Deadline: 29 March 2026

Important Guidelines for Students

- **Individual Work:** This is an individual assignment.
- **Code Integrity:** Ensure your code is neatly indented and includes comments.
- **Warning:** If there is a syntax error, zero marks will be awarded for that part.
- **Demo Readiness: Make sure your code is fully functional and running during the demo.**
- **Submission**: ONLY `.ipynb` and `Game tree (Handwritten + Simulation)` must be submitted in a zipped folder
- **Plagiarism:** There is a strict zero-tolerance policy for cheating or AI-generated logic.
- **Programming Language:** Python
- **No Email Submissions:** Under NO circumstances will submissions via email or any other way be accepted. Only soft copy submission on GCR.

Submission Checklist

- [ ] File named as **`i241234_Moaz_A2.zip`**.
- [ ] Clear textual code snippets which can be run on our systems (no screenshots of code).
- [ ] All output images and plots clearly visible.
- [ ] ONLY `.ipynb` and `Game Tree` are submitted in a zipped folder.
- [ ] If you have simulated on ROS or any other platform, then upload the video of simulation on drive and add a text file with link of that video in zip file.

GAME AI – UNO using Adversarial Search

Implement a simplified **UNO game with 3 players** and develop an **AI agent using minimax + Expectimax search** to choose the best move.

Game Setup

Player Configurations

The game consists of three players with distinct logic:

- **Player 1 (AI - Minimax):** Must implement a **Defensive Strategy**. Focus on preventing opponents from winning and managing "Skip" cards effectively.
- **Player 2 (AI - Expectimax):** Must implement an **Offensive Strategy**. Focus on aggressive card shedding and calculating the probability of the "Draw" deck.
- **Player 3 (User/AI):** This player will have two modes and user will be able to choose the mode.
    - **Manual Mode:** The user provides input for moves i.e., the user will be able to play.
    - **Simulation Mode:** Player 3 acts as an AI using a **Minimax** function (can reuse Player 1 logic). The entire game will be simulated step by step and the moves of all players must be shown on the screen. User will just be an observer watching the three AI players play the game.

Initial cards:

Each player gets 5 cards

Deck contains cards:

Red 0–9
Blue 0–9
Green 0–9
Yellow 0–9
Skip cards

Note: To keep the game simple, there are no special cards other than skip.

Game Rules (Simplified)

Students must implement these rules:

Rule 1 — Valid Move

A player may play a card if:

same color
OR
same number

Example:

Top card is: Red 5

Valid cards are:

Red 3
Blue 5
Red Skip etc

Rule 2 — Drawing Card

If no valid card exists:

player must draw 1 card

Rule 3 — Skip Card

If player plays **Skip**:

next player's turn is skipped

Rule 4 — Winning

Player wins when:

cards = 0

Game stops.

Game State Representation

Students must define:

```
state = {
  ai_cards
  opponent1_cards
  opponent2_cards
  top_card
  deck
}
```

Evaluation Function

Use the following scoring baseline for your search algorithms:

$$Score = 50 - 5(C_{AI}) + 2(C_{opp}) + 3(S)$$

- C_{AI}: Cards in the current player's hand.
- C_{opp}: Average cards held by the two opponents.
- S: Number of Special (Skip) cards in hand.

***Note**: You must tune these weights differently for "Offensive" vs "Defensive" strategies.*

Search Algorithms

- **Minimax (Defensive):** Used by P1. It assumes opponents play to minimize the current player's score.
- **Expectimax (Offensive):** Used by P2. It treats the "Draw Card" action as a **Chance Node**, calculating the expected value based on the remaining deck and probability of occurrence of cards.

Expectimax Algorithm

Students must implement:

MAX node → AI turn
CHANCE node → draw card
Opponent node → opponent turn

Pseudo-code:

```
function expectimax(state, depth):

 if terminal OR depth == 0
    return evaluation(state)

 if AI turn
    return max(expectimax(child))

 if chance node
    return Σ P(child)*expectimax(child)

 if opponent turn
    choose random legal move
```

Example Game Tree

Students should generate tree like this below. This is not the exact or correct tree. This is just a sample.

```
                AI
          /     |     \
       move1  move2  Draw
        |       |       |
        P2      P2    Chance
       / \     / \       |
   play draw  play draw  cards and so on...
```

Output Example

Program should print:

Top card: Red 7

AI hand:
Red 3
Blue 7
Red Skip

AI decision(All possible decisions considered at depth 1):

Play: Red 3
Expected score: 43

Play: Green Skip

Expected score: 48.2

Additional Implementation Requirements

Students must implement:

1. Card Class

card = (color, number)

---

2. Deck Generator

Shuffle deck.

---

3. Legal Move Generator

get_valid_moves(hand, top_card)

---

4. State Transition

apply_move(state, move)

---

5. Expectimax Search

Depth: 3

---

6. Minmax Search

Depth: 3

Deliverables

Students must submit:

1- Source code
2- Game simulation output
3- Generated search tree (example run)
4- Explanation of evaluation function in **ipynb**
5- Picture of Game tree (handwritten) that you will make before implementation. It should have at least levels where each level comprises of either Player 1,2,3 or chance nodes.
6- Github repository link is required in the file. Make sure to upload on GitHub and sync, so that we can make sure that the work done is yours, through GitHub history.
7- **Bonus**: Simulate this game in ROS OR make a proper GUI on any simulation environment, platform, or language. GUI simulation has a bonus of 10 and ROS simulation has a bonus of 20.

Marking Scheme (It can change slightly)

Component	**Marks**
Game rules implementation	15
Minmax(Defensive)	15
Expectimax algorithm (Offensive)	15
Evaluation function	15
Game tree generation through simulation	10
Game tree (Handwritten for both algorithm)	10
Code quality	15
Simulation Output & GitHub History	20
Comparison of both Algorithms Ensure the concluding section of the `.ipynb` file addresses key aspects such as:	10

Component	Marks
● The strategy employed. ● Which algorithm performed best. ● The reasoning behind the superior performance of that algorithm.	
Bonus: GUI or simulation in ROS	20
Total:	125 + Bonus marks